

Распределённая система обработки пользовательских запросов

Курс «System Design»

2. Проверка связи

Меня хорошо видно, слышно?

3. О себе

Шелепов Григорий

Архитектор

4. Цель и задачи проекта

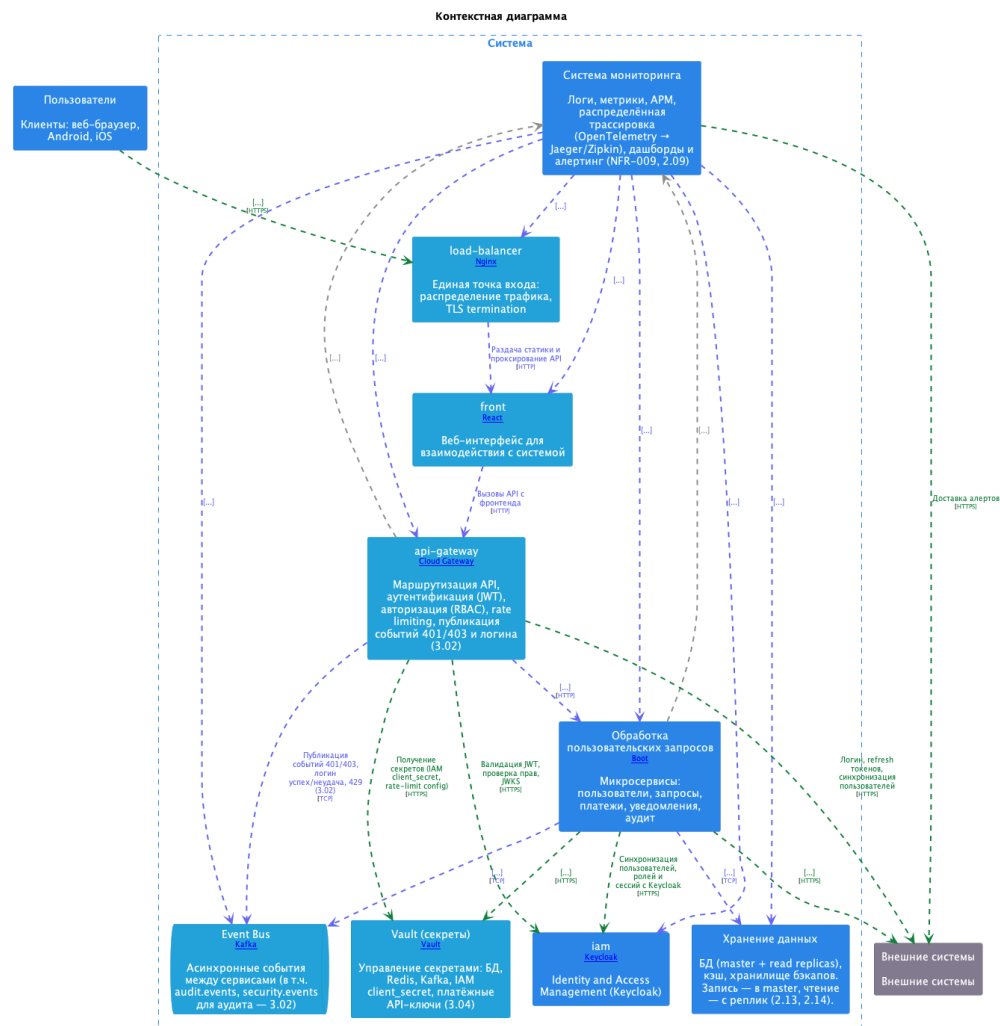
Цель — спроектировать архитектуру высоконагруженной системы.

Задачи

1. Зафиксировать функциональные и нефункциональные требования, риски.
2. Описать контекст, контейнеры, потоки данных, модель данных, API (OpenAPI), хранение (PostgreSQL, Redis).
3. Спроектировать ИБ системы (OAuth 2.0 / OIDC / JWT, RBAC, Vault, аудит).
4. Спроектировать мониторинг (метрики, логи, трассировка по NFR-009).
5. Задать развёртывание IaC (Terraform, Ansible), контейнеры и Kubernetes, CI/CD.

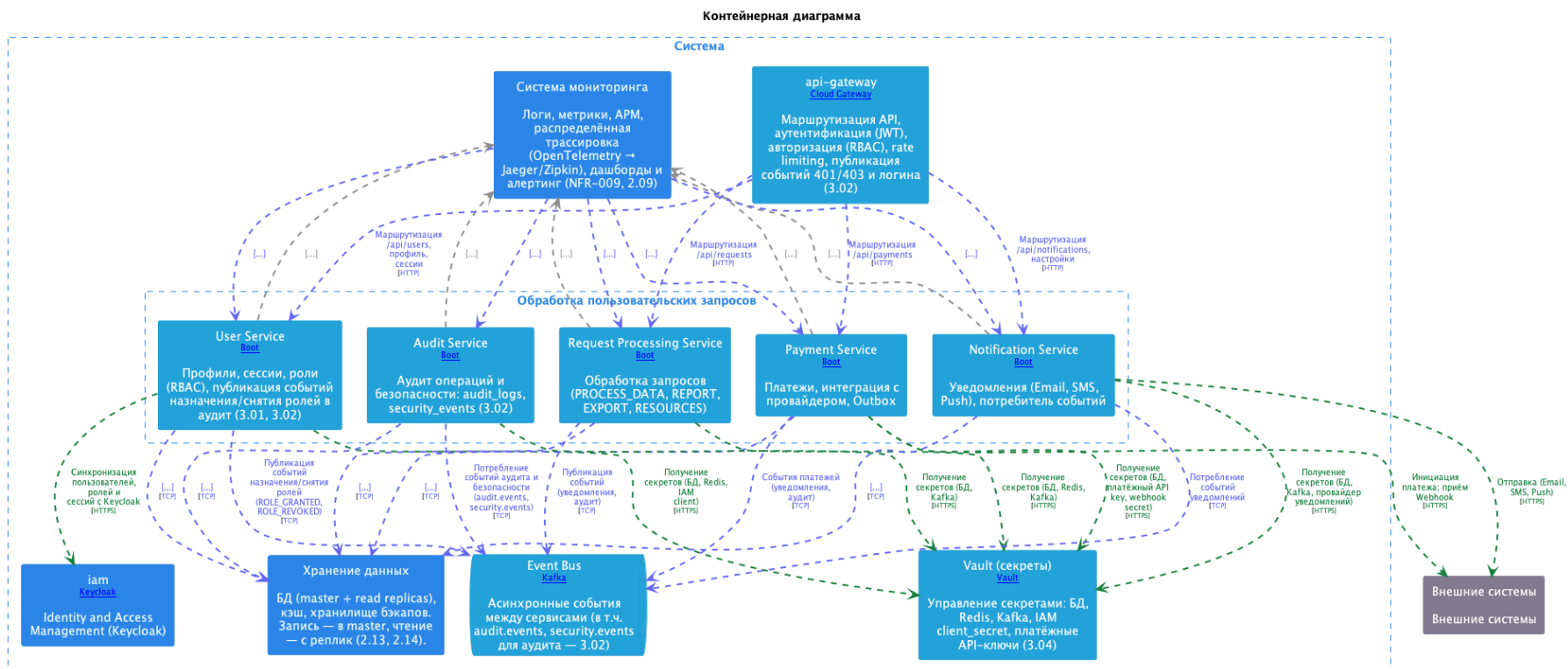
5. Контекст системы

Система взаимодействует с пользователями (веб, мобильное приложение), внешними платёжными и провайдерами уведомлений, IAM; внутренние сервисы изолированы за API Gateway.



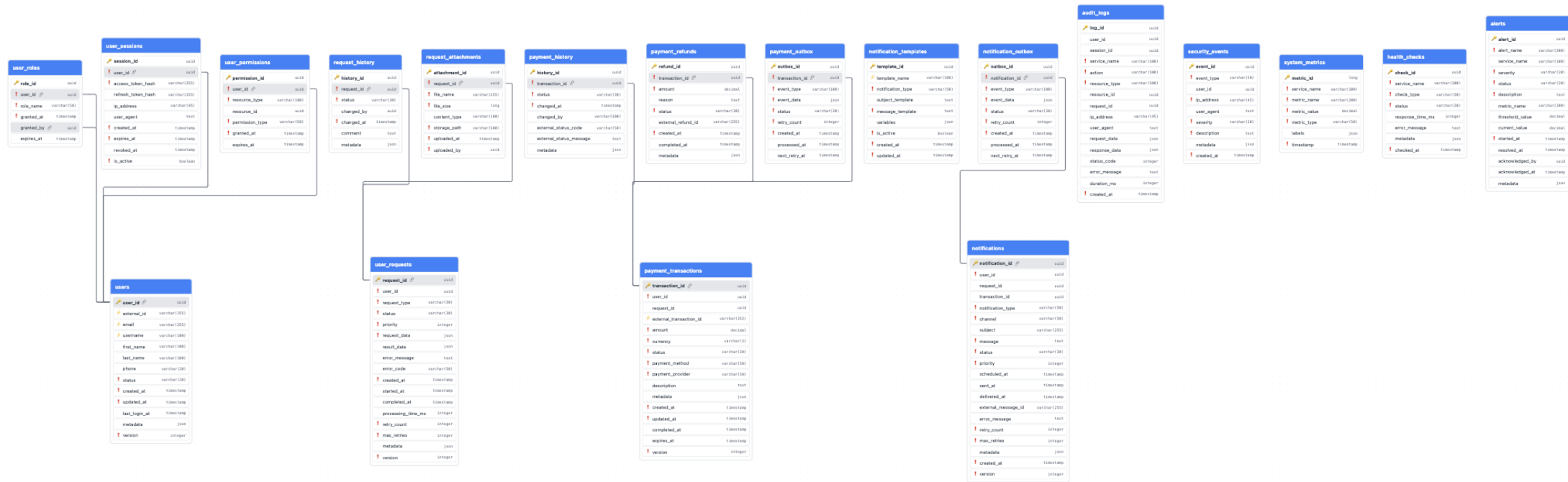
6. Контейнерная архитектура backend

API Gateway, доменные микросервисы, шина событий, кэш, интеграции с IAM и внешними API.



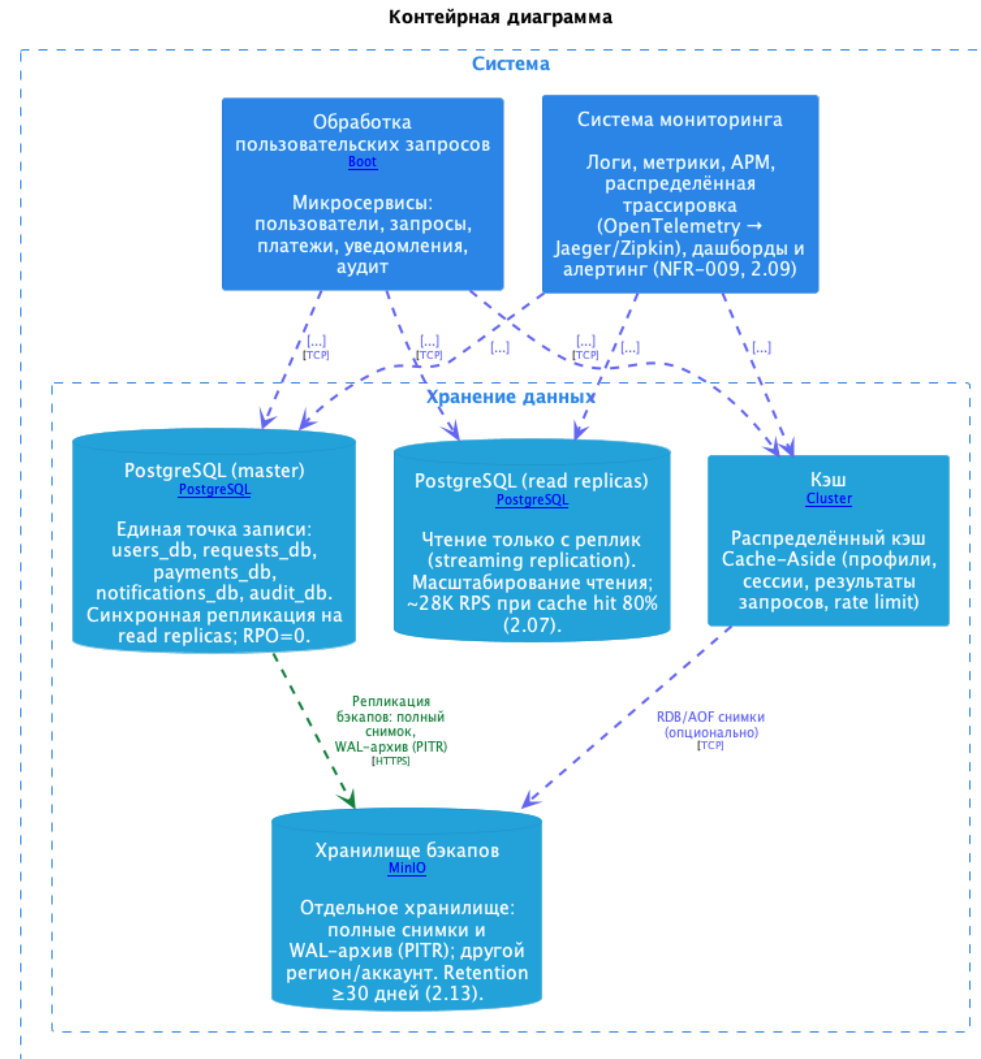
7. Модель данных

Логическая модель по доменам: пользователи, запросы, платежи, уведомления.



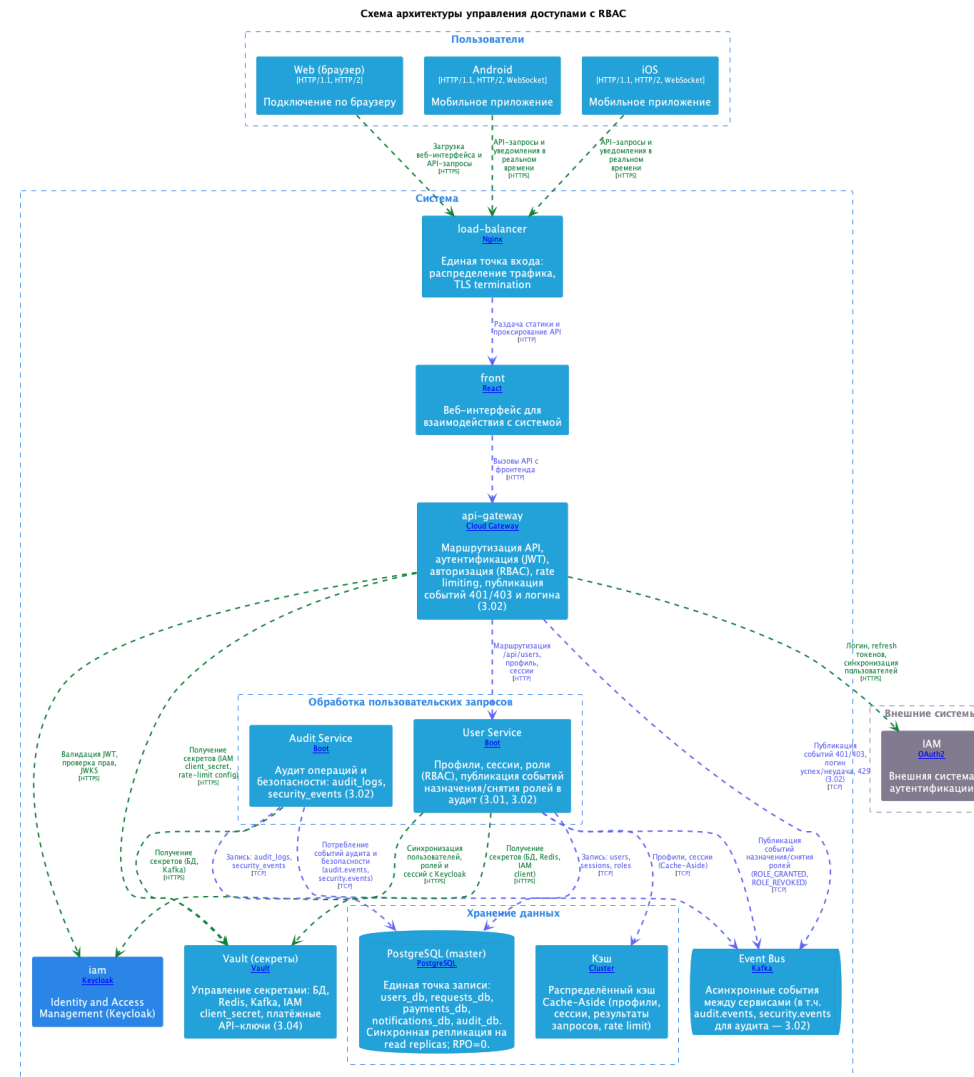
8. Хранение и кэш

PostgreSQL (master и read replicas), Redis, объектное хранилище бэкапов.



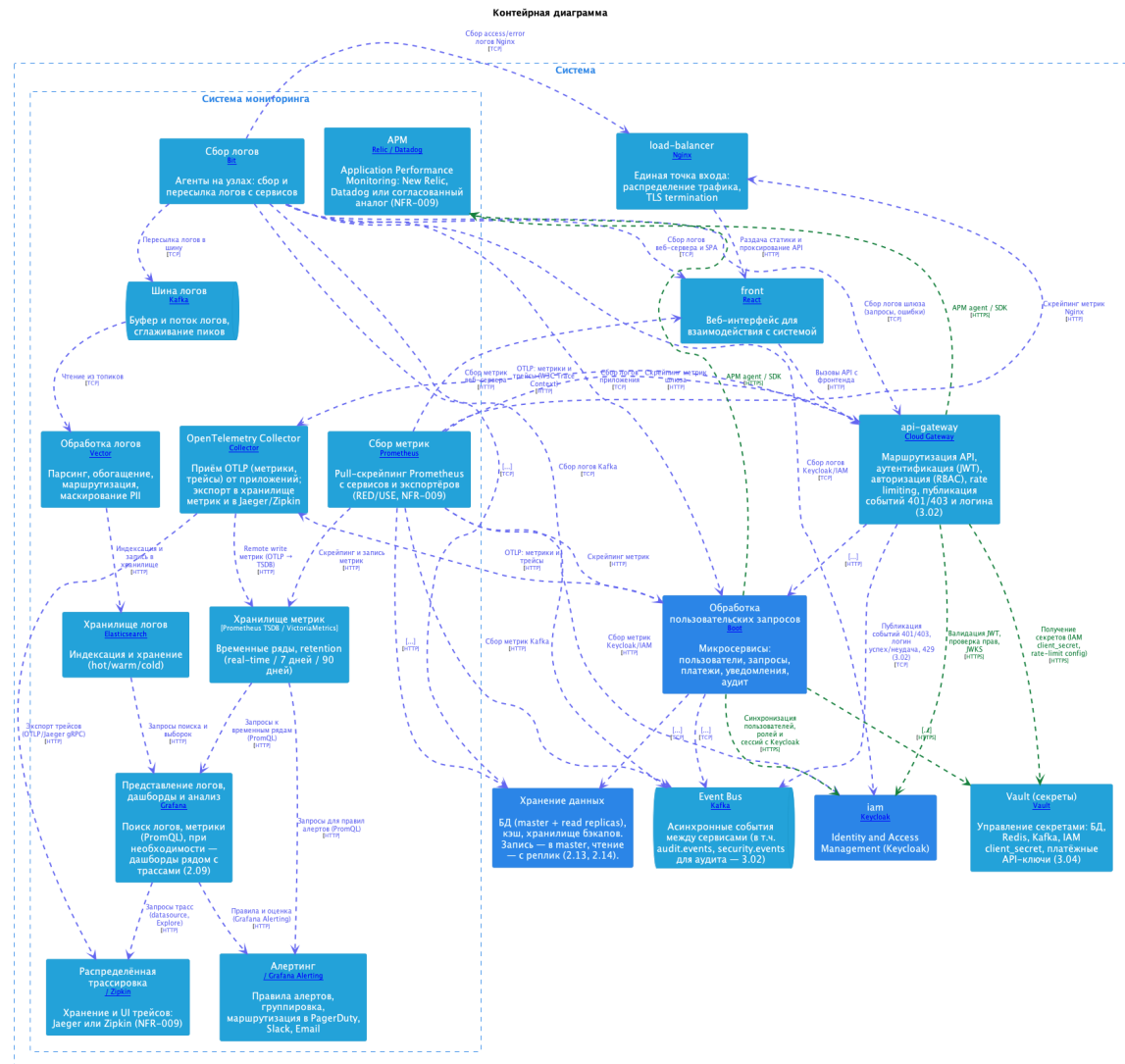
9. Безопасность и управление доступом

Keycloak (IAM), RBAC с проверкой владельца ресурса, Vault для секретов, события безопасности и аудит.



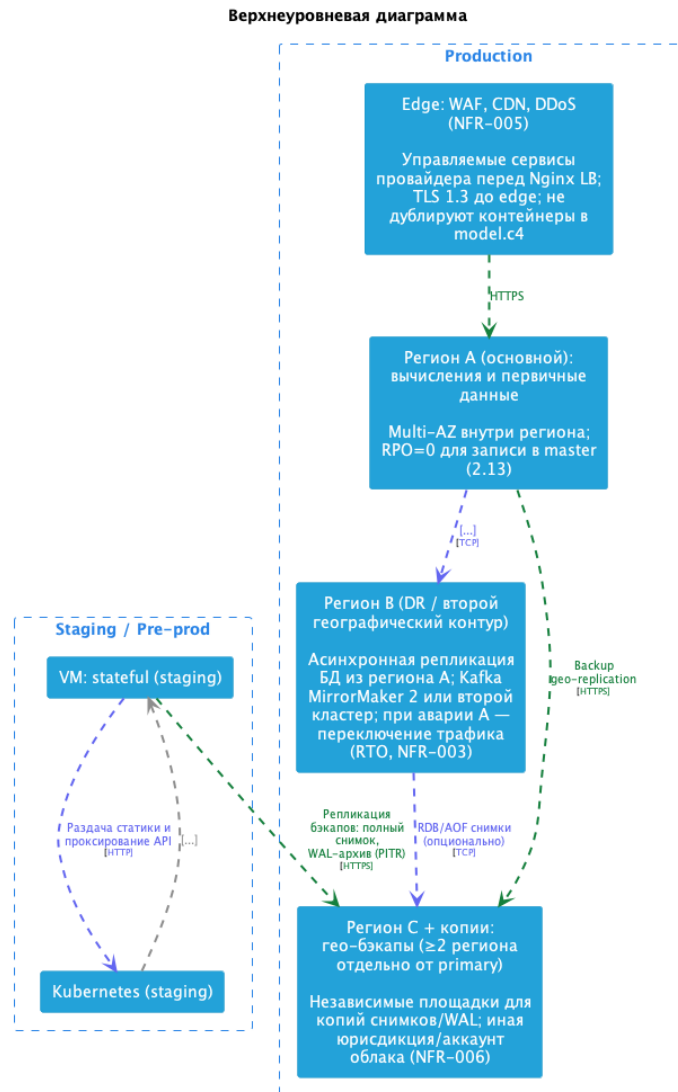
10. Observability

Сбор метрик (Prometheus), дашборды и алерты (Grafana, Alertmanager), логи и трассировка.



11. Развёртывание

Зоны edge, Kubernetes и VM, DR и гео-бэкапы; stateful-сервисы вне кластера или managed по модели; манифесты `infra/k8s/`, Terraform, Ansible.



12. Технологии

В проекте (репозиторий, инструменты и конфигурации)

- Моделирование и схемы: **LikeC4**, экспорт в **PlantUML**, публикация на **GitHub Pages**; тесты контейнеров и связей (**Vitest**).
- Контракты и данные: **OpenAPI 3** (`openapi/api-gateway.yaml`), **DBML** (`dbml/model.dbml`), экспорт диаграммы в PNG.
- Безопасность как код: **Keycloak** (realm, клиенты, роли), **HashiCorp Vault** (политики ACL по сервисам, примеры engine).
- Инфраструктура и поставка: **Terraform** (AWS: сеть, ALB, EC2, S3, IAM), **Ansible** (Docker на VM, PostgreSQL 15), **Dockerfile** для backend, **Kubernetes** + **Kustomize** (`infra/k8s/`), пайплайн **GitLab CI** (сборка, `kubectl kustomize`, деплой).
- Прикладной контур: микросервисы на **Spring Boot**, **API Gateway**, синхронные и асинхронные вызовы; при необходимости — **Kafka** как шина событий.
- Данные: **PostgreSQL** (master / read replicas), **Redis** (кэш), объектное хранилище бэкапов; политики масштабирования и резервирования.
- Идентификация и секреты в рантайме: **OAuth 2.0 / OIDC / JWT**, **Keycloak**, **Vault**.
- Observability: **Prometheus**, **Grafana**, **Alertmanager** (манифесты в кластере); ещё **Fluent Bit**, **Vector**, **Elasticsearch**, **Loki**; трассировка **OpenTelemetry** → **Jaeger / Zipkin**; APM (**New Relic / Datadog** или аналог).

13. Итого

Выводы — цель проекта достигнута: зафиксированы требования, архитектура, данные, безопасность, мониторинг и контур поставки; репозиторий содержит воспроизводимые конфигурации.

14. Вопросы

???

15. Спасибо!

Спасибо за внимание!